Building Multi-Agent System

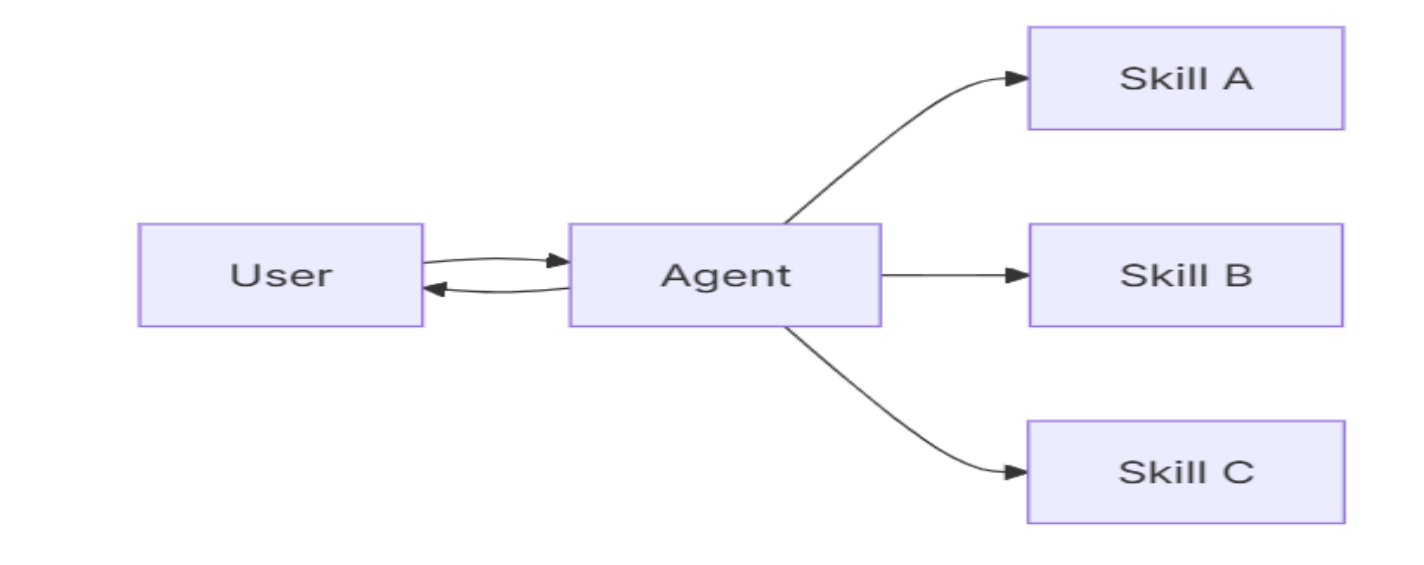
Skills - Implementation in LangChain

What it is:

- Specialized capabilities packaged as invokable "skills" that augment agent's behavior
- Primarily prompt-driven specializations that agent can invoke on-demand
- Progressive disclosure of specialized prompts and domain knowledge
- Single agent with many possible specializations

When to use:

- Want single agent with many possible specializations
- Don't need to enforce specific constraints between skills
- Different teams need to develop capabilities independently
- Common examples: coding assistants (skills for different languages), knowledge bases (skills for different domains), creative assistants (skills for different formats)



Building Multi-Agent System

Skills - Implementation in LangChain

What it is:

- Tool loads specialized skill prompt on-demand
- Skills become available based on context or user needs
- Lightweight composition—simpler than full sub-agents

How it works:

- **@tool load_skill** - Tool to load specialized prompts
- **skill_name parameter** - Select which skill to load
- **Load from file/database** - Retrieve skill content dynamically
- **system_prompt lists skills** - Agent knows available skills
- **Progressive disclosure** - Load only when needed, not upfront

```python
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def load_skill(skill_name: str) -> str:
    """Load a specialized skill prompt.

    Available skills:
    - write_sql: SQL query writing expert
    - review_legal_doc: Legal document reviewer

    Returns the skill's prompt and context.
    """
    # Load skill content from file/database
    ...

agent = create_agent(
    model="gpt-4.1",
    tools=[load_skill],
    system_prompt=(
        "You are a helpful assistant. "
        "You have access to two skills: "
        "write_sql and review_legal_doc. "
        "Use load_skill to access them."
    ),
)
```

Building Multi-Agent System

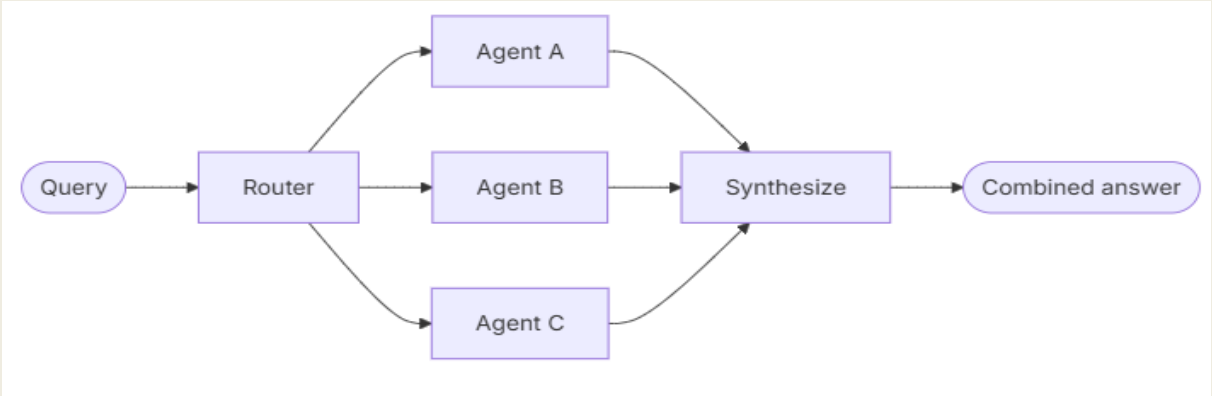
Router - Implementation in LangChain

What it is:

- Routing step classifies input and directs it to specialized agents
- Router decomposes the query
- Zero or more specialized agents invoked in parallel
- Results synthesized into coherent response

When to use:

- Have distinct verticals (separate knowledge domains that each require their own agent)
- Need to query multiple sources in parallel
- Want to synthesize results into combined response



Downsides of Orchestrator:

Higher latency — sequential step-by-step execution vs router's parallel fan-out
Single point of failure — orchestrator goes down, entire workflow stalls
More LLM calls — planning + re-planning + monitoring burns more tokens and cost
Added complexity — needs state management, plan tracking, and feedback loops
Harder to debug — multi-step stateful flows are trickier to trace than a simple route-and-merge
Tighter coupling — agents become dependent on orchestrator's plan structure and sequencing

Building Multi-Agent System

Multiple Agents (Parallel)

What it is:

- Router sends query to multiple agents in parallel
- Use Send for parallel fan-out to multiple agents
- Results synthesized into combined response

How it works:

- **classify_query()** - Returns list of agents to invoke
- **Send(agent, state)** - Send query to each agent
- **Parallel execution** - All agents run simultaneously
- **Results collected** - Synthesized into final answer

```python
from langgraph.types import Send

def route_query(state: State) -> list[Send]:
    """Route to multiple agents in parallel."""
    agents = classify_query(state["query"])  # Returns list of agents

    # Send to multiple agents in parallel
    return [Send(agent, state) for agent in agents]
```

Building Multi-Agent System

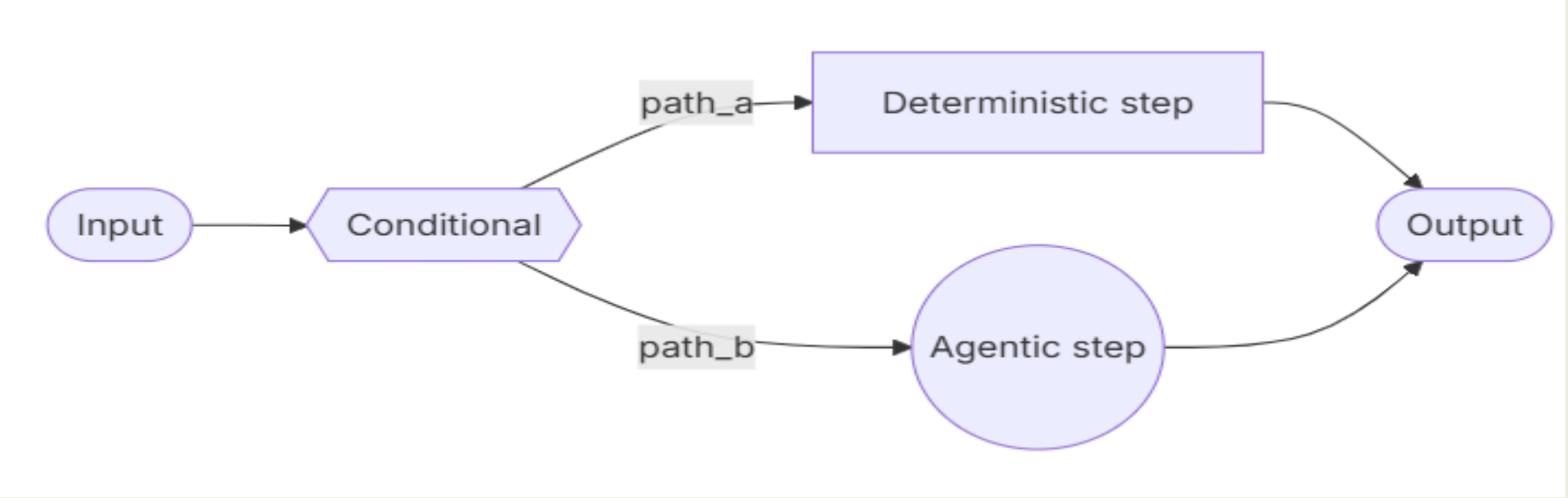
Custom Workflow - Implementation in LangChain

What it is:

- Define your own bespoke execution flow using LangGraph
- Complete control over graph structure
- Mix deterministic logic with agentic behavior
- Support for sequential steps, conditional branches, loops, and parallel execution

When to use:

- Standard patterns (subagents, skills, router) don't fit your requirements
- Need to mix deterministic logic with agentic behavior
- Use case requires complex routing or multi-stage processing
- Want to embed other patterns as nodes in your workflow



Building Multi-Agent System

Custom Workflow - Implementation in LangChain

What it is:

- Call LangChain agent directly inside any LangGraph node
- Combine flexibility of custom workflows with convenience of pre-built agents
- Each node can be simple function, LLM call, or entire agent with tools

How it works:

- **create_agent()** - Create LangChain agent with tools
- **agent_node()** - LangGraph node that invokes agent
- **agent.invoke()** - Call agent with query
- **StateGraph(State)** - Define workflow graph
- **add_node()** - Add agent as node in workflow
- **add_edge()** - Define flow between nodes (START → agent → END)
- **compile()** - Compile workflow for execution

```python
from langchain.agents import create_agent
from langgraph.graph import StateGraph, START, END

agent = create_agent(model="openai:gpt-4.1", tools=[...])

def agent_node(state: State) -> dict:
    """A LangGraph node that invokes a LangChain agent."""
    result = agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]
    })
    return {"answer": result["messages"][-1].content}

# Build a simple workflow
workflow = (
    StateGraph(State)
    .add_node("agent", agent_node)
    .add_edge(START, "agent")
    .add_edge(END, "agent")
    .compile()
)
```

Agent Observability

- **What is observability?**
  - Observability is the ability to understand a system's internal behavior **just by looking at its outputs**.
  - To make a system observable:
    - **Instrument the code:** Add monitoring tools so the system can generate information such as Metrics, Logs, Traces, Events.
    - **Data correlation and context:** Combine data from various entities to:
      - Establish a correlation
      - Spot patterns
      - Identify unusual behavior
    - **Incident response:** When something breaks:
      - Alerts are generated
      - The right team is notified
      - They can act quickly to fix the issue
    - **AIOps:** Machine learning models automatically aggregate, correlate, and prioritize incident data.
      - Filter out unrelated data and create an alert signal or detect issues that can impact the system's performance and efficiency.

